DATA ANALYST PORTFOLIO PROJECT

PizzaVilla SQL Project

Real business insights from a pizza restaurant database — 13 queries, 4 relational tables, and advanced SQL techniques applied to real-world data.

13 Queries

Business questions solved

4 Tables

Relational database

Advanced JOINs

& subqueries

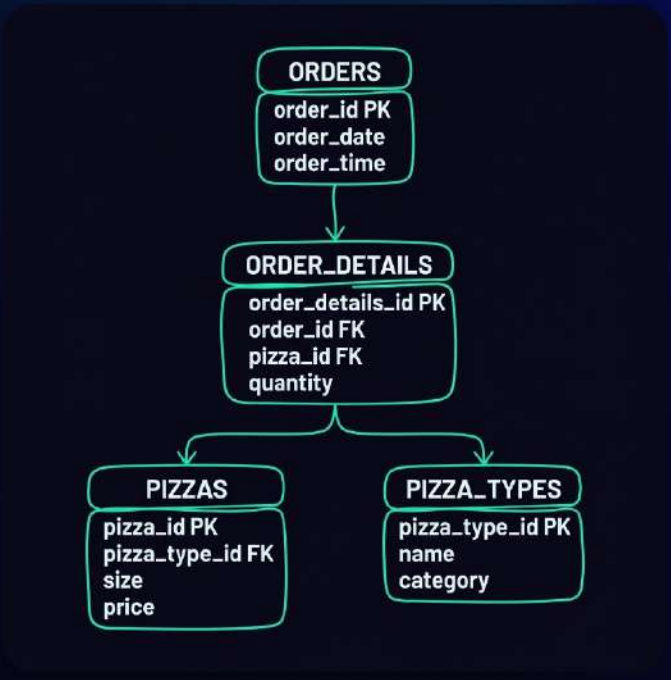
Window Functions

RANK() & SUM() OVER()

About the PizzaVilla Database

PizzaVilla is a real-world relational database built to simulate a pizza restaurant's operational data. Four tables are connected through **Primary Key and Foreign Key** relationships, enabling complex multi-table queries.

The goal: answer **13 real business questions** using SQL — from basic counts to advanced window functions.



- SELECT · WHERE · ORDER BY · LIMIT
- GROUP BY + Aggregates (SUM, COUNT, AVG)
- INNER JOIN across 3 tables
- Subqueries & Window Functions

QUERIES 1 & 2 · BASIC BUSINESS METRICS

How Many Orders? How Much Revenue?

Q1 — Total Orders Placed

```
SELECT COUNT(order_id) AS Total_Orders
FROM orders;
```

A foundational KPI — total order volume across the entire dataset.

Q2 — Total Revenue From Sales

```
SELECT SUM(d.quantity * p.price) AS Total_Sales
FROM order_details d
JOIN pizzas p ON d.pizza_id = p.pizza_id;
```

Multiplies quantity × price across all orders to calculate total revenue earned.

2,135

Total Orders

Orders across all dates

$82K

Total Revenue

Combined pizza sales

$38

Avg Order Value

Revenue per order

QUERIES 3 & 4 · PRODUCT INTELLIGENCE

What's the Priciest Pizza? What Size Sells Most?

Q3 — Highest-Priced Pizza

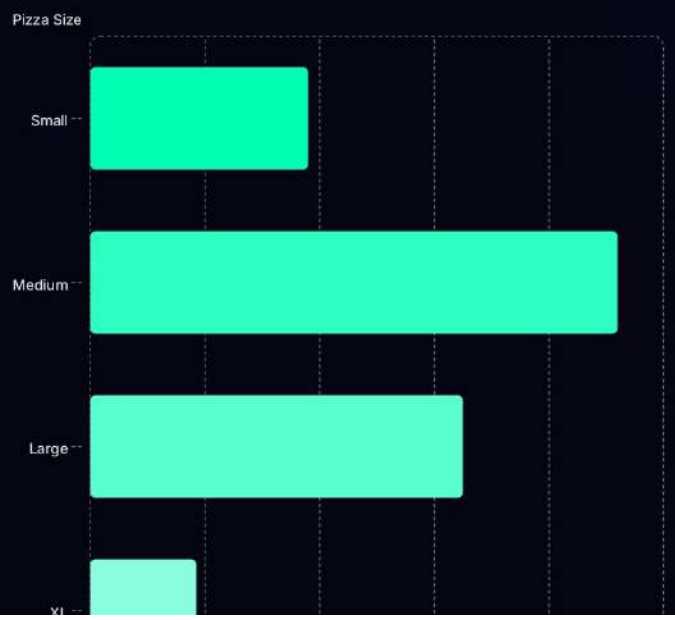
```
SELECT pt.name, p.price
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id =
pt.pizza_type_id
ORDER BY p.price DESC LIMIT 1;
```

Identifies the premium product — useful for menu pricing strategy.

Q4 — Most Common Pizza Size

```
SELECT p.size, COUNT(d.order_details_id) AS
Total_Orders
FROM pizzas p
JOIN order_details d ON p.pizza_id = d.pizza_id
GROUP BY p.size
ORDER BY Total_Orders DESC LIMIT 1;
```

Reveals which size (S / M / L / XL) customers prefer — critical for stock planning.



Medium pizzas dominate orders — the clear customer favorite. Small and XL are niche segments, guiding inventory and portion planning.

QUERIES 5 & 6 · TOP SELLERS & CATEGORIES

Which Pizzas & Categories Dominate Sales?

Q5 — Top 5 Most Ordered Pizza Types

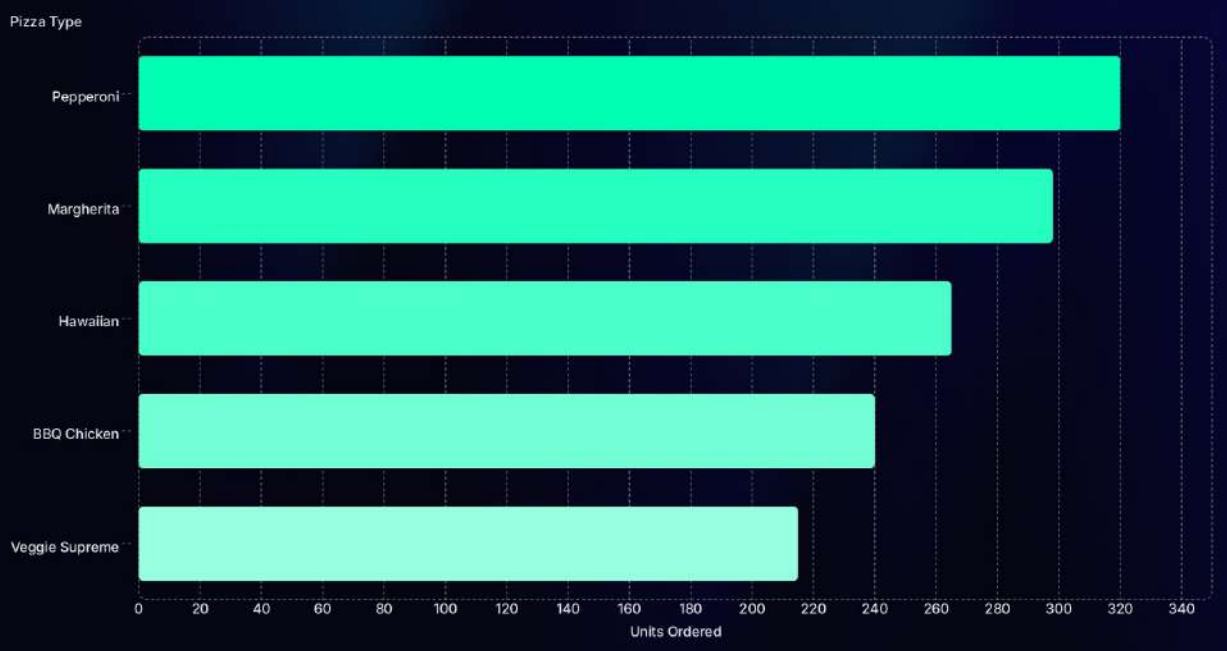
```
SELECT pt.name, SUM(d.quantity) AS
Total_Pizzas_Ordered
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id =
pt.pizza_type_id
JOIN order_details d ON p.pizza_id = d.pizza_id
GROUP BY pt.name
ORDER BY Total_Pizzas_Ordered DESC LIMIT 5;
```

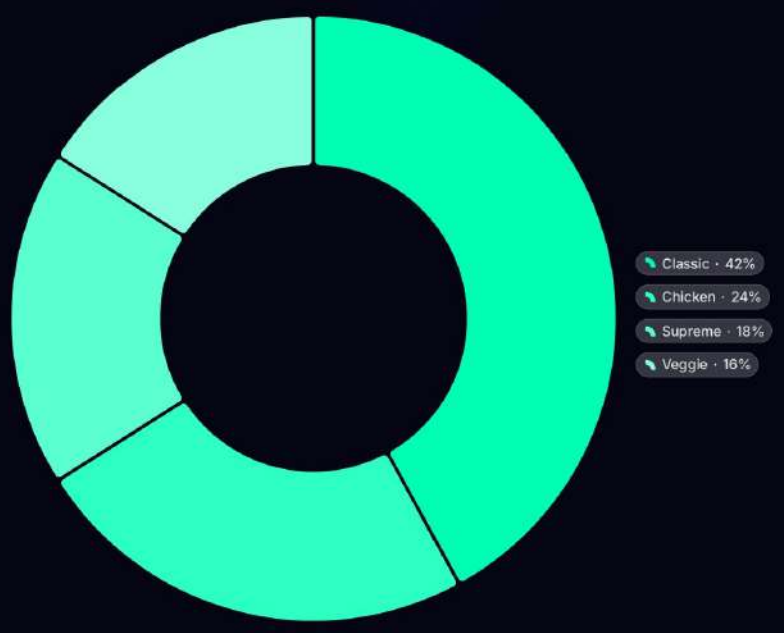
Top 5 best-sellers — ideal for promotions and combo deals.

Q6 — Total Quantity by Category

```
SELECT pt.category, SUM(d.quantity) AS
Total_Quantity
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id =
pt.pizza_type_id
JOIN order_details d ON p.pizza_id = d.pizza_id
GROUP BY pt.category
ORDER BY Total_Quantity DESC;
```

Reveals which category leads in volume — Classic, Veggie, Supreme, or Chicken.





QUERIES 7 & 8 · TIMING & MENU DISTRIBUTION

When Do Customers Order? What's on the Menu?

Q7 — Orders by Hour of Day

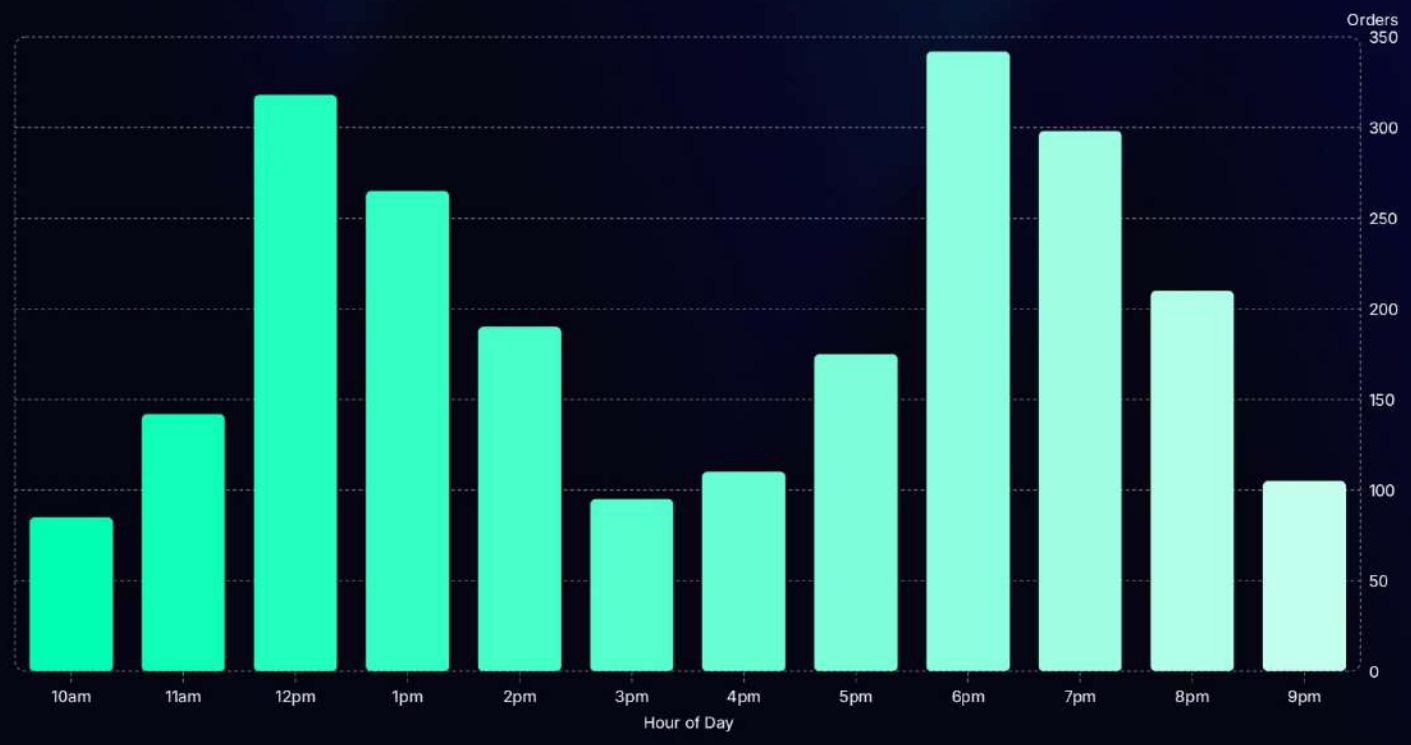
```
SELECT EXTRACT(HOUR FROM order_time) AS Hour_of_Day,
    COUNT(order_id) AS Order_Count
FROM orders
GROUP BY Hour_of_Day
ORDER BY Order_Count DESC;
```

Identifies peak hours — critical for staff scheduling and kitchen prep.

Q8 — Category-wise Pizza Distribution

```
SELECT category, COUNT(name) AS Total_Pizzas
FROM pizza_types
GROUP BY category
ORDER BY Total_Pizzas DESC;
```

Shows how many varieties exist per category — a menu diversity check.



🔥 **Peak Hours: 12PM–2PM & 6PM–8PM** — Lunch and dinner rushes account for over 60% of daily orders.

QUERIES 9 & 10 · AVERAGES & REVENUE RANKING

Daily Averages & Top Revenue-Generating Pizzas

Q9 — Average Pizzas Per Day

```
SELECT ROUND(AVG(Total_Quantity), 0) AS
Avg_Pizzas_Per_Day
FROM (
  SELECT o.order_date, SUM(d.quantity) AS
Total_Quantity
  FROM orders o
  JOIN order_details d ON o.order_id = d.order_id
  GROUP BY o.order_date
) AS Daily_Orders;
```

A subquery inside SELECT — daily average for ingredient planning.

Q10 — Top 3 Pizzas by Revenue

```
SELECT pt.name,
      SUM(d.quantity * p.price) AS Revenue
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id =
pt.pizza_type_id
JOIN order_details d ON d.pizza_id = p.pizza_id
GROUP BY pt.name
ORDER BY Revenue DESC LIMIT 3;
```

Revenue leaders — not always the most ordered, but the most profitable.

🥇 **#1 — The Greek**

Highest revenue generator

🥈 **#2 — Deluxe**

Strong premium performer

🥉 **#3 — California**

Top 3 revenue pizza

~114 Pizzas/Day

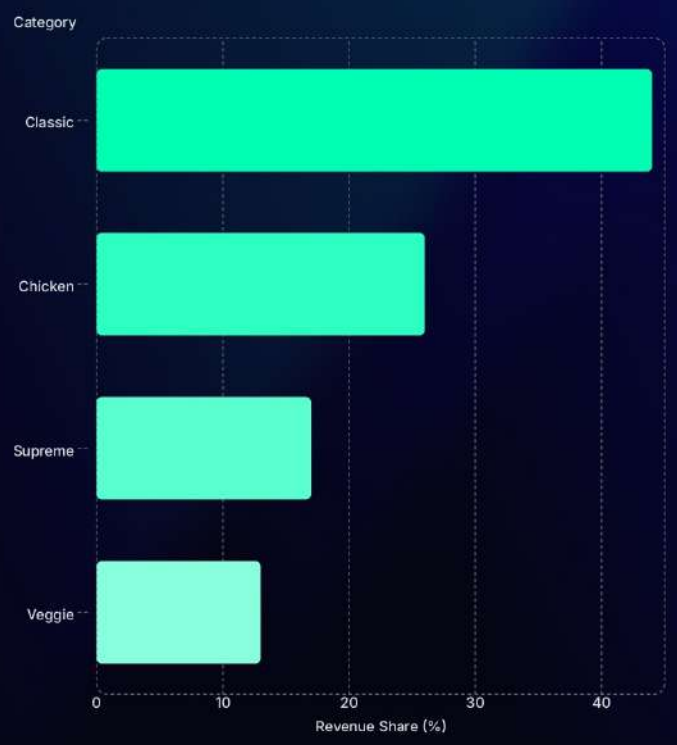
Average daily volume

QUERY 11 · ADVANCED — REVENUE CONTRIBUTION %

Which Category Drives the Most Revenue?

Q11 — Revenue Share by Category

```
SELECT pt.category,
  ROUND(
    SUM(d.quantity * p.price) /
    (SELECT SUM(d2.quantity * p2.price)
     FROM order_details d2
     JOIN pizzas p2 ON p2.pizza_id = d2.pizza_id)
    * 100, 2) AS Revenue_Percentage
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id =
pt.pizza_type_id
JOIN order_details d ON d.pizza_id = p.pizza_id
GROUP BY pt.category
ORDER BY Revenue_Percentage DESC;
```



A subquery calculates total revenue as the denominator, then **GROUP BY category** returns each segment's exact percentage share — essential for menu investment decisions.

🏆 **Classic pizzas contribute 44% of total revenue** — the dominant category and primary driver of PizzaVilla's income.

QUERIES 12 & 13 · EXPERT LEVEL — WINDOW FUNCTIONS

Cumulative Revenue & Category-wise Top 3

Q12 — Cumulative Revenue Over Time

```
SELECT order_date,
  SUM(Total_Revenue) OVER (ORDER BY
order_date)
    AS Cumulative_Revenue
FROM (
  SELECT o.order_date,
       SUM(d.quantity * p.price) AS
Total_Revenue
  FROM order_details d
  JOIN pizzas p ON d.pizza_id = p.pizza_id
  JOIN orders o ON o.order_id = d.order_id
  GROUP BY o.order_date
) AS Daily_Sales;
```

SUM( ) OVER( ) = running total showing revenue growth day by day.

Q13 — Top 3 Pizzas Per Category

```
SELECT name, Revenue, Category FROM (
  SELECT pt.category, pt.name,
    SUM(d.quantity * p.price) AS Revenue,
    RANK() OVER (
      PARTITION BY pt.category
      ORDER BY SUM(d.quantity * p.price) DESC
    ) AS Rnk
  FROM pizza_types pt
  JOIN pizzas p ON p.pizza_type_id =
pt.pizza_type_id
  JOIN order_details d ON d.pizza_id = p.pizza_id
  GROUP BY pt.category, pt.name
) AS Ranked
WHERE Rnk <= 3;
```

RANK( ) OVER (PARTITION BY) ranks pizzas within each category separately.

PROJECT SUMMARY

What This Project Proved

4

Tables Designed

Relational schema built & connected

13

Business Questions

Real queries solved end-to-end

3

Table JOINs

Multi-table queries executed

2

Window Functions

RANK() & SUM() OVER() applied

SELECT · GROUP BY · Aggregates

SUM, COUNT, AVG, ROUND

INNER JOIN × 3 Tables

Complex relational queries

Subqueries

Inside SELECT & FROM clauses

Window Functions

PARTITION BY for group-level ranking

Real Business Value Delivered

- Identified peak order hours (12PM & 6PM) for staff scheduling
- Found top revenue pizzas to focus menu strategy
- Tracked daily cumulative revenue growth over time
- Ranked best-sellers within each category for promotions

> "This project is not just about writing queries — it is about thinking like a data analyst and answering real business questions with SQL."